SDI Investment Property Marketplace

Property Media Lifecycle

Upload, assignment, maintenance and purge

Document type	Operational requirements. What is built, and what is not, is section 9.
Build at	`v0.9.11`, github.com/neilgreene/property-management
Covers	Photographs and other listing media, from arrival to destruction
Does not cover	Documents (fee agreements, inspection reports) — a different retention regime

This page is intentionally blank.

Contents

Photography is not an attachment. It is a publication. A photograph of the front of a house identifies the property as surely as its street address, so putting one on a listing is the same act as disclosing the address — and the platform's entire commercial model is that the address is withheld until a fee agreement is signed. Every requirement in this document follows from that.

1. The problem this document exists to solve

A photograph lives in two places at once: as bytes on a filesystem, and as a row in `core.property_media` that says what those bytes mean. Neither is useful alone. The bytes without the row are an orphan nobody can find; the row without the bytes is a broken image on a live listing.

Almost every operational failure in a media system is these two drifting apart. A file is copied in and never registered. A row is deleted and the file is left behind, quietly consuming storage for years. A database is restored from Tuesday and a filesystem from Wednesday. Somebody clears space by hand and removes the wrong thing, because nothing on disk said what it was for.

So the requirements below are mostly about keeping the two in agreement, and about being able to answer two questions at three in the morning:

The question	Who asks it
I am looking at this file. What is it, and may it be deleted?	Whoever is clearing disk, restoring a backup, or responding to a takedown demand
I am looking at this listing. Where are its files?	Whoever is fixing a broken image, re-encoding a set, or archiving a sold property

A layout that cannot answer both, quickly and without a running application, will be worked around — by hand, badly, under time pressure.

2. Where a file lives, and how to find it

2.1 The layout

Four zones on the shared mount, each with one job. The mount is storage, not a web root: nothing here is served by path.

```
/srv/media/
  inbox/
    SDI-1009 - Columbus OH - Single Family 5bd 2ba/
        README.txt          <- what this property is, written by the system
        IMG_4471.jpg        <- dropped from a PC. Any filename.
    _unsorted/              <- photographs whose property is not yet known
  store/
    SDI-1009/
        7f3a91c4e8b2....-orig.jpg     <- media_id, then variant
        7f3a91c4e8b2....-720.jpg
  quarantine/               <- failed ingest, kept for diagnosis
  purged/                   <- deleted, awaiting retention expiry
```

2.2 Why the filename is the media_id

This is the answer to the maintenance question, and it is worth being explicit about. In `store/`, the filename **is** the primary key of the database row. Not a slug, not a sequence, not the original camera filename — the `media_id` itself.

That means anybody holding a file can identify it with one query and no guesswork, and the answer includes everything needed to decide its fate:

```
$ psql -d sdi -c "SELECT listing_ref, caption, position, is_primary,
                        reveals_location, published_at, deleted_at
                   FROM core.property_media m
                   JOIN core.property p USING (property_id)
                  WHERE media_id = '7f3a91c4-e8b2-...'"
```

And the reverse direction needs no query at all — `ls store/SDI-1009/` is the complete set of files for a listing. That redundancy is deliberate: the listing reference in the path is there so a human can navigate, and the database is authoritative if the two ever disagree.

> **Requirement. The row stores its own relative path in a `storage_path` column rather than reconstructing it from the listing reference. A listing reference can be corrected; a path that is derived from one silently stops resolving when it is, and the failure appears weeks later as a broken image nobody can explain.**

2.3 Why the inbox is separate from the store

Because a human drop zone and a machine-managed store have opposite requirements. The inbox must tolerate anything — a phone filename, a duplicate, a HEIC, a screenshot, a file that is still copying. The store must contain only files that have been validated, re-encoded, stripped of metadata and registered. Mixing them means the store contains unvalidated bytes, and the whole guarantee collapses.

A file is removed from the inbox once it is in the store. An inbox that stays full is a queue that stopped, and it should be visible as one.

3. Getting photographs in

Two routes, because they serve different moments. Both converge on one processing pipeline — if they do not, the weaker one becomes the way everything gets in.

	Drop on the share	Upload in the panel
Suits	Forty photographs from a shoot	One photograph, right now
Who	Anyone with the share mounted	Signed-in staff
Knows the property?	By folder	By being on its page
Attribution	Filesystem owner — weak	Authenticated person — exact

3.1 What both routes must do

Every arriving file, by either route, passes through the same steps in the same order:

- **Sniff the content, do not trust the extension.** A `.jpg` that is not a JPEG is either a mistake or an attack, and both end in quarantine.
- **Reject what is not an image**, and cap the size. A 60 MB RAW file is a mistake, not a listing photo.
- **Re-encode.** This is what strips metadata, and it is not optional here — see 3.2.
- **Generate the thumbnail** that `thumb_url` points at. Twenty-five full-size photographs on one page is 9.4 MB against 2.1 MB of thumbnails.
- **Hash the content** and refuse an exact duplicate already on the listing. The same photo arriving twice is the normal case, not the rare one.
- **Register the row** — pending, unpublished, and `reveals_location` defaulted to **true**.

3.2 Metadata stripping is not housekeeping

A photograph taken on a phone carries GPS coordinates in its EXIF block. Accurate to a few metres. Upload one untouched and the exact location of the property is sitting inside the file, readable by anything, for the one thing the entire platform exists to withhold.

> **The gate would still be intact in the database and completely bypassed in practice. It also carries the camera serial, and often the photographer's name — which is a data protection question on top of a commercial one. Re-encoding on ingest is the control. It has to sit in the pipeline, not in a checklist, and it has to be on the share path as well as the upload path, because a file copied from a PC never passed through a browser.**

3.3 Fail closed on arrival

A newly arrived photograph is assumed to reveal the location until a human says otherwise. The alternative — assume it is safe, publish, and correct later — means

the window between arrival and review is a window in which the gate is open. Corrections do not un-disclose anything.

The cost of failing closed is that somebody has to click. That is the intended cost.

4. Assigning meaning

A file in the store is inert. It becomes part of a listing when somebody decides what it is. That decision is the publication act, and it is where the interface earns its keep.

4.1 What has to be decided per photograph

Decision	Effect	Default
Which property	Everything else. Wrong here and a photo of one house sells another	Folder, or unset
KEY image	The card thumbnail — the one image most people ever see	None until chosen
Order	Gallery sequence	Arrival order
Shows the street	The gate. True means investors behind the fee only	**True**
Caption	What the viewer is told it is	Empty
Published	Whether it appears at all	No

4.2 The gate, exposed as a checkbox

“Shows the street” is `reveals_location`, and it deserves to be phrased in those words rather than as a technical flag. The person ticking it is deciding who may see the photograph, and they should be able to tell that from the label alone.

> **The flag means identifying, not exterior. A photograph of a different house — the representative stock images currently on every listing — identifies nothing and is not gated. A photograph of this house's front door, with a number on it, is gated whether or not the street is in frame. The interface should say which test is being applied, because the obvious reading of the words is the wrong one.**

4.3 Requirements on the panel

- **Every photograph on one screen**, thumbnails, with pending ones visually distinct from published ones. Somebody must be able to see at a glance that six photos arrived and none are live.
- **Setting the KEY image is one click**, and moves the flag off whatever held it. There is exactly one, enforced by a unique index, so the interface must make that visible rather than let a save fail.
- **Reordering is dragging.** Typing position numbers is how galleries end up with two photos at position 3.
- **Publish is deliberate and plural.** Review six, publish six, one action, one audit entry.
- **Unassigned photographs are surfaced, not hidden.** The `_unsorted/` queue is the normal result of a bulk download, and it needs a screen that shows them and asks which property.
- **Staff authorisation is checked in the database** on every call. A hidden button is not a permission.

5. Managing what is already there

5.1 The ordinary operations

Operation	What it must not do
Recaption	Nothing subtle. Free text, audited
Reorder	Leave two photographs claiming one position
Change the KEY image	Leave the listing with none, even for an instant
Replace a photograph	Silently keep serving the old bytes from a cache
Move to another listing	Leave the file in the old listing's folder
Re-gate (tick or untick)	Take effect only on the next deploy

5.2 The two that are harder than they look

Moving a photograph between listings changes its `storage_path`, which means a file move and a row update that must not half-happen. If the move succeeds and the update fails, the listing has a broken image and the file is an orphan in a folder it does not belong to. It is one transaction, with the filesystem move last and reversible.

Un-gating a photograph — ticking a gated photo as safe — is a disclosure that cannot be withdrawn. Anyone who was shown it has it. This is the one operation that deserves a confirmation naming what is about to become public, and it is the one most likely to be done by accident while tidying.

5.3 Correcting a mistake

The likeliest serious error is a photograph attached to the wrong property: an investor is looking at a house that is not the one they are buying. The system should make this cheap to fix and hard to do — which means the assignment step shows the property's city, type and size next to the photograph, so an obvious mismatch is obvious.

6. Purge: when photographs are destroyed

6.1 What triggers it

Trigger	Urgency	Scope
Staff removes a photograph	Routine	One file
Listing sold or withdrawn	Scheduled	The listing's set
Licence expired or revoked	Dated in advance	Everything under that right
Takedown demand from a copyright holder	**Hours**	Named files, everywhere
Location metadata found after publication	**Immediate**	One file plus its derivatives

Trigger	Urgency	Scope
Duplicate or superseded	Housekeeping	One file

The middle two are the ones that make a purge mechanism worth building. `gov.data_right` already records expiry dates and termination terms; a right that expires with photographs still published is a licence breach the system created by doing nothing.

6.2 Deletion happens twice

Unpublish is immediate and reversible: the row is marked, the photograph leaves every listing, the authorising route stops serving it. **Destruction** is later and final: the file moves to `purged/`, waits out a retention window, and then the bytes are removed.

A single-step delete is wrong in both directions. It makes the routine case unrecoverable — the wrong tick removes a photograph nobody can get back — while making the urgent case no faster, because the urgent part is stopping publication, not reclaiming disk.

6.3 A purge is not complete when the file is gone

A takedown demand names an image, not a file. Satisfying it means every copy: the original, the thumbnail, any cache, any CDN edge, and any browser holding it under a `Cache-Control` header the system set itself. A one-day cache means a one-day tail, and that has to be a known number rather than a surprise.

> **And backups are the honest problem. A photograph purged today is still in last night's backup, and in the monthly archive, and possibly in an offsite copy on a different retention schedule. No purge reaches those without destroying the backup. The policy must state what it does about that — the usual answer is that backups age out on their own schedule and are not restored selectively — but it must state it, because "we deleted it" and "it is gone" are different claims and only one of them is true.**

6.4 Legal hold beats retention

If a property is in dispute, its photographs are evidence and must not be destroyed on schedule. A hold flag suspends purge and is released deliberately. Without it, a well-behaved retention job destroys exactly the material somebody later needs, having done precisely what it was told.

7. Maintenance and operations

7.1 Reconciliation, nightly

The job that keeps the two stores honest. It reports, and does not fix:

Finding	Usual cause	Why not auto-fix
Row with no file	Interrupted ingest, restore mismatch	Deleting the row hides a restore that failed
File with no row	Ingest crashed after write	It may be the only copy of something
File in `purged/` past retention	Normal	This one *is* safe to automate
Inbox older than a day	Ingest stopped	Needs a person to notice
Published photo under an expired right	Licence lapsed	Unpublishing is a business decision

A reconciliation report nobody reads is the same as no reconciliation. It belongs next to the governance banner already on the admin screen, where staff are looking anyway.

7.2 Backup and restore

The filesystem and the database must be restorable to the same moment, or reconciliation will find hundreds of discrepancies that are really one mistake. In practice that means the media backup runs immediately after the database dump, and both are labelled with the same timestamp.

> **The restore is the thing to rehearse. A media store that has never been restored is a media store of unknown value, and the moment you find out is the moment it matters.**

7.3 What clearing disk by hand must look like

It will happen. Somebody will be at a full filesystem at an unreasonable hour. The layout in section 2 is what makes it survivable: `purged/` is always safe to empty, `quarantine/` is always safe to empty, `inbox/` is a queue and must not be touched, and `store/` is never cleared by hand — every file in it is referenced by a row, and the way to remove one is to purge it.

7.4 Growth

Roughly half a megabyte per photograph including its thumbnail, eight photographs per listing, so about 4 MB per property and 4 GB at a thousand listings. Not a capacity problem. It becomes one the day somebody uploads video, which is why the ingest cap and the accepted content types are limits worth setting deliberately rather than discovering.

8. Who may do what

	Add	Assign / reorder	Un-gate	Unpublish	Destroy
Admin	yes	yes	yes	yes	yes
Agent (own listings)	yes	yes	**no**	yes	no
Integration / scanner	yes	no	no	no	no
Investor	no	no	no	no	no

Two deliberate asymmetries. **An agent cannot un-gate**: releasing a location-revealing photograph is a disclosure decision about the whole platform's model, not a listing-level edit. **Nobody but an admin destroys bytes**, because destruction is the one action with no undo.

The scanner can create and nothing else. A process that ingests unattended should not be able to publish, and the fail-closed default in 3.3 is what makes that true rather than merely intended.

8.1 What must be audited

Every event that changes what the public can see: registered, assigned, published, un-gated, unpublished, purged. Who, when, and from where. A photograph appearing on a listing is a publication, and a publication with no record of who authorised it cannot be defended when somebody asks.

9. What exists today

So this document is not mistaken for a status report.

Capability	Today
`core.property_media` with the gate	**Built.** Row policy, `reveals_location`, enforced by RLS
`thumb_url` and the thumbnail split	**Built.** Cards use the small file, detail uses the full one
Photographs on listings	**Built,** from files committed to the repository
Provenance recorded	**Built,** as `STOCK-PHOTOGRAPHY` — and **unreviewed**, source unestablished
Shared mount	**Built.** A host path, not a named volume — `down -v` must not destroy photographs
Authorising route	**Built.** `/media/file/<media_id>` re-asks the database as the caller; not visible is a 404, not a 403
Ingest, EXIF stripping, quarantine	**Built.** `scan-media.js`, with a test asserting GPS is gone from the stored bytes
Pending on arrival, gated by default	**Built.** Nothing is published by arriving
Unpublish, purge, retention, legal hold	**Built.** Two-step deletion; a hold beats a due retention date
Reconciliation	**Built.** Reports both drift directions, fixes neither

Capability	Today
Media audit trail	**Built.** `core.media_event`
The properties panel	**Not built.** The operations exist as API functions; there is no screen
Self-describing folders, `_unsorted` triage	**Partly.** `_unsorted` is counted and reported; nothing creates the folders
Browser upload	**Not built.** The share is the only way in

Note what is still true of the seeded images. The twenty-five photographs committed to the repository are still served statically from `web/public/assets`, by path, to anyone who guesses one. That is acceptable only because they are representative stock images with nothing to leak. Anything genuinely location-revealing has to arrive through the store, where the route decides who gets bytes — and the seeded set should move there before it is mistaken for a pattern to follow.

9.1 Suggested order

Step	Delivers	Unblocks
1	Mount, store layout, authorising route, ingest with EXIF stripping	**Done.** Real photography can exist at all
4	Unpublish, purge, retention, legal hold, reconciliation	**Done.** The obligations in sections 6 and 7
2	The properties panel: assign, KEY image, gate, publish	**Next.** Staff stop needing a developer
3	Self-describing folders, `_unsorted` triage	Bulk drops from a PC

Step 4 came early rather than last because its pieces — two-step deletion, retention, legal hold — are cheap to write alongside the schema and expensive to retrofit onto a store already holding a year of photographs under rules it was not built for.

Step 2 is the gap that matters now. Every operation a person needs exists as a callable function with its authorisation enforced in the database, and there is no screen that calls them. Until there is, publishing a photograph means somebody at a psql prompt — which is the situation the store was built to end.

Requirements as of `v0.9.11`. Regenerate after either the requirements or the build changes: `python3 docs/generate_media_lifecycle.py`.